

BigBasket Bangladesh

Automation Testing

Project Type: Web Application Automation Testing

Member 1

Name: Nasib-ul Islam
ID: 232-35-286

Member 2

Name: Sabbir Hossain
ID:232-35-494

Member 3

Name: Tasnim Chowdhury
ID: 232-35-020

BigBasket Bangladesh

BigBasket is an e-commerce / grocery shopping web application. **Our project focuses on testing important user-facing functionalities through browser automation.**

Why This Application?

E-commerce applications contain repetitive and important user workflows, making them highly suitable for automation testing — high transaction volume, frequent regression cycles, and critical revenue-driving flows.

Main User Activities Tested



Open the website



Search for products



View product details



Add products to cart



Validate login behaviour



Verify search results

Testing Objectives and Scope

Testing Objectives

- 1 Automate repetitive web testing activities
- 2 Reduce manual testing effort
- 3 Improve test execution consistency
- 4 Validate important user workflows
- 5 Detect UI/functional issues efficiently
- 6 Generate reusable test reports

Scope — Included

- ✓ Homepage
- ✓ Product Search
- ✓ Product Details
- ✓ Add to Cart
- ✓ Login validation
- ✓ Search result validation

Testing Type

Functional Testing

UI Testing

Regression-oriented Automation Testing

COVERAGE BREAKDOWN

Modules or Features Automated

Module	Automation Coverage
Homepage	Website opening / validation
Search	Product search
Product Details	Name, price, image, availability
Cart	Add product to cart
Login	Invalid login validation
Search Results	Result availability

Search Test Data

-  **Milk**
-  **Rice**
-  **Biscuit**
-  **Egg**
-  **Cooking Oil**

Tools and Technologies Used



Programming Language

Python



Automation

Selenium WebDriver ·
webdriver-manager



BDD

Behave · Gherkin



Design Pattern

Page Object Model (POM)



Configuration

ConfigParser



Reporting

behave-html-formatter ·
HTML Report



Version Control

Git · GitHub

Test Scenarios and Test Cases

1

Invalid Login

Enter invalid email/password → Click Login → Verify error message.

2

Product Details

Search "Milk" → Open first product → Verify name, price, image, availability.

3

Add Product to Cart

Search "Rice" → Open first product → Add to cart → Verify successful addition.

4

Product Search

Search Milk, Rice, Biscuit, Egg, Cooking Oil → Verify results are displayed.

Overall Test Coverage

7

Feature Files

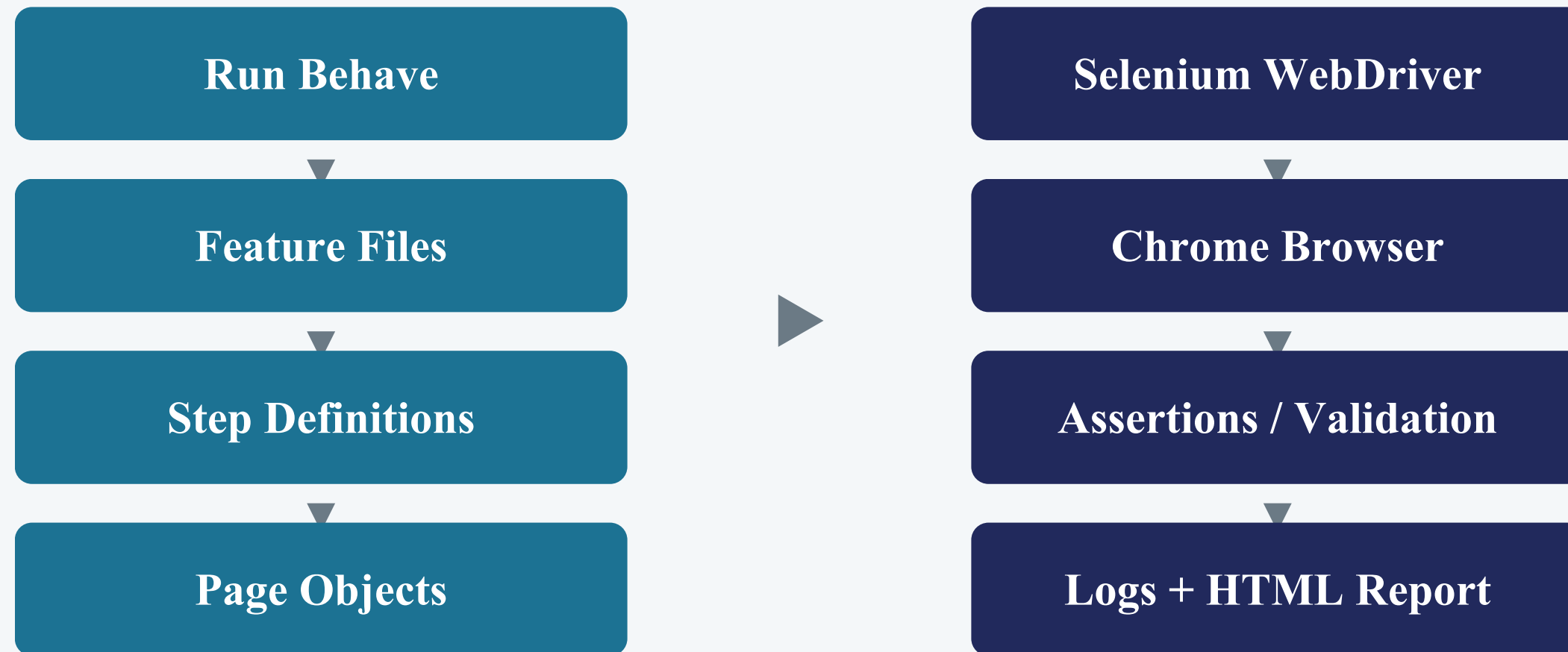
22

Scenarios

82

Steps

Selenium Automation Workflow

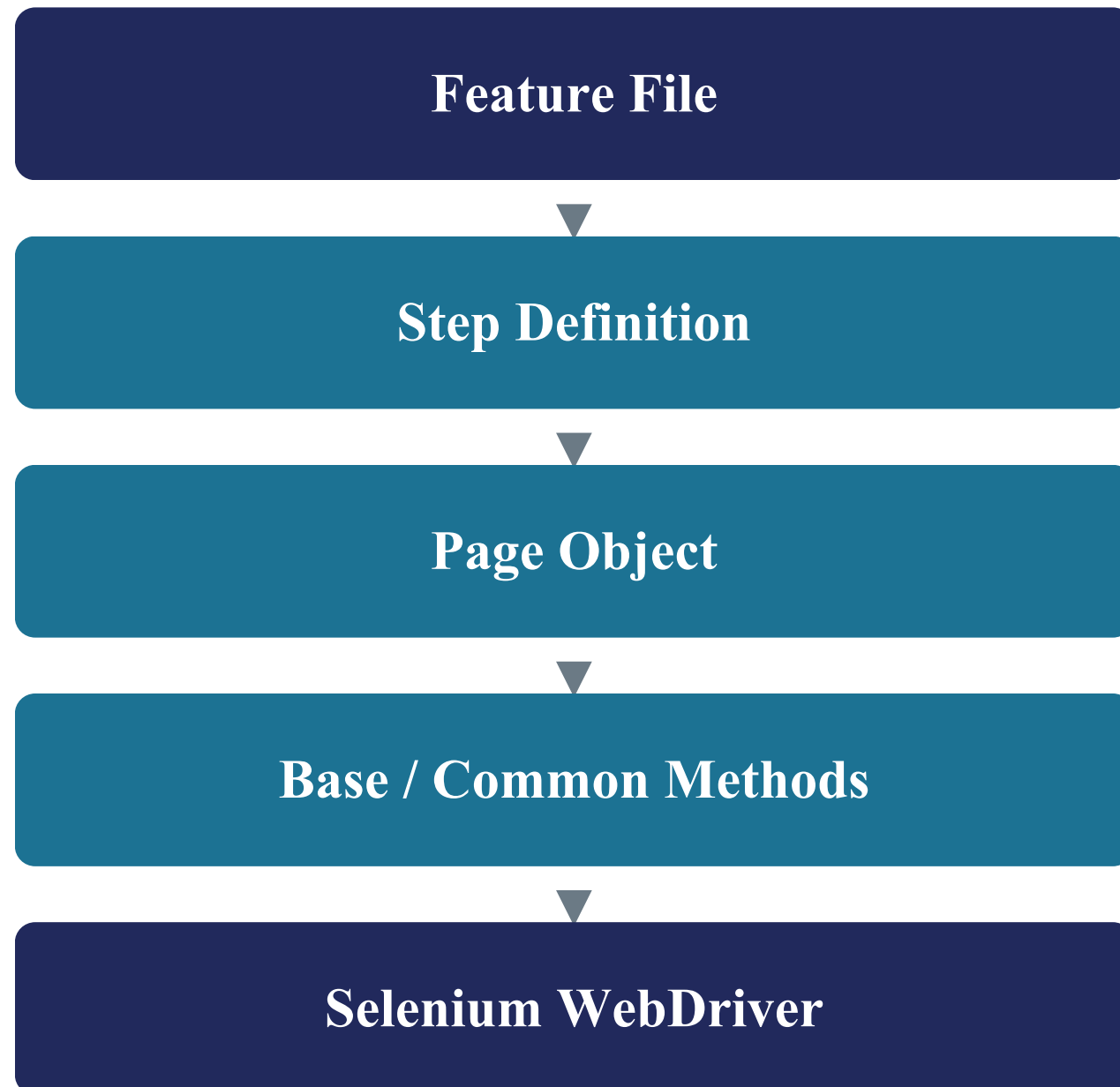


EXAMPLE



Page Object Model (POM) Structure

POM separates test behaviour from webpage locators and actions.



Advantages

- ✓ Reusable code
- ✓ Easier maintenance
- ✓ Easier locator management
- ✓ Less code duplication
- ✓ Better readability

Example

Instead of writing Selenium actions directly inside every test, page-specific actions are maintained inside the relevant Page Object — keeping test logic clean and locators centralized.

Screenshots of Important Code Sections

Screenshots-01: BDD Feature File

```
Scenario: Search for Milk  
  When the user searches for "Milk"  
  Then search results should be displayed
```

Test scenarios are written using Gherkin syntax with Given, When and Then.

Screenshots-02: Step Definition

```
@when('the user searches for "{product}"')
def step_search_product(context, product):
    context.search = SearchPage(context.driver)
    context.search.search_product(product)

@then("search results should be displayed")
def step_verify_search_results(context):
    assert context.search.verify_search_results()

@when("the user opens the first product")
def step_open_first_product(context):
    context.search.open_first_product()
```

Sabbir Hossain,

Step Definitions connect Gherkin steps with Python automation code.

Screenshot 3: Page Object Method

```
def search_product(self, product):  
    search_box = self.find_element(self.SEARCH_BOX)  
  
    search_box.clear()  
    search_box.click()  
    search_box.send_keys(product)  
    search_box.send_keys(Keys.ENTER)  
  
    WebDriverWait(self.driver, 15).until(  
        EC.presence_of_element_located(self.PRODUCT_LIST)  
    )
```

Page Objects contain locators and reusable actions for interacting with the webpage.

Screenshot 4: WebDriver Setup

```
def before_scenario(context, scenario):

    logger.info("-----")
    logger.info(f"Starting Scenario: {scenario.name}")

    try:
        context.driver = DriverFactory.get_driver()

        # Page Objects
        context.home = HomePage(context.driver)
        context.search = SearchPage(context.driver)
        context.product = ProductPage(context.driver)
        context.cart = CartPage(context.driver)

        logger.info("Chrome WebDriver initialized successfully")

    except Exception as error:
        logger.error(
            f"Failed to initialize WebDriver: {error}",
            exc_info=True
        )
        raise
```

WebDriver setup manages browser initialization and test execution lifecycle.

```
Sabbir Hossain, 4 days ago • Final BigBasket Automation Project
def after_scenario(context, scenario):

    try:
        if scenario.status == "failed":
            logger.error(
                f"Scenario FAILED: {scenario.name}"
            )

            if hasattr(context, "driver") and context.driver:
                ScreenshotUtils.capture(
                    context.driver,
                    scenario.name.replace(" ", "_")
                )

                logger.info(
                    f"Failure screenshot captured for: {scenario.name}"
                )

            else:
                logger.info(
                    f"Scenario PASSED: {scenario.name}"
                )

        except Exception as error:
            logger.error(
                f"Error during after_scenario: {error}",
                exc_info=True
            )

        finally:
            if hasattr(context, "driver") and context.driver:
                try:
                    context.driver.quit()

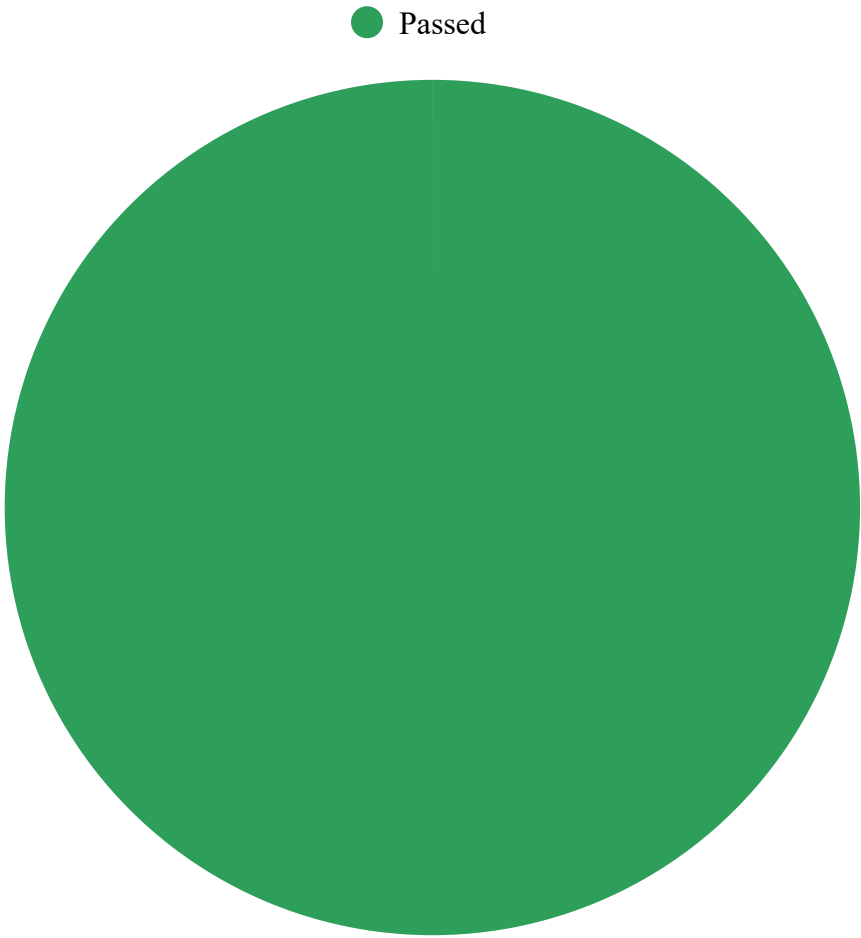
                    logger.info(
                        f"Browser closed: {scenario.name}"
                    )

                except Exception as error:
                    logger.error(
                        f"Error while closing browser: {error}",
                        exc_info=True
                    )
```

Test Results — Passed, Failed & Skipped

Result	Count
Features Passed	7
Features Failed	0
Features Skipped	0
Scenarios Passed	22
Scenarios Failed	0
Scenarios Skipped	0
Steps Passed	82
Steps Failed	0
Steps Skipped	0

Scenario Outcome



100% Scenario Pass Rate | Execution Time: ~1 minute 28 seconds

Project Output / Report Screenshots

Generated Output:
reports/behave_report.html

Screenshot 1 — Test Execution Result

```
2026-08-16 05:04:04,599 | INFO | builtins | BigBasket BDD Test Execution Finished
2026-08-16 05:04:04,599 | INFO | builtins | =====
7 features passed, 0 failed, 0 skipped
22 scenarios passed, 0 failed, 0 skipped
82 steps passed, 0 failed, 0 skipped
Took 1min 28.407s
(venv) sabhirhossain@Sabhirs-MacBook-Air: BigBasketAutomation % open reports/behav
```

Complete Behave test execution with final pass/fail/skip summary.

Screenshot 2 — HTML Test Report

Behave Test Report

Features: passed: 7
Scenarios: passed: 22
Steps: passed: 82
Finished in 88.4 seconds

[Expand All](#) | [Collapse All](#) | [Expand All Failed](#)

Generated HTML report showing the automated test execution results.

Screenshot 3 — Detailed Scenario Result

Feature: BigBasket Bangladesh Product Search		
Background:		
Scenario: Search for Milk		features/search.feature:6
Given the user opens the BigBasket Bangladesh website	(0.746s)	features/steps/homepage_steps.py:6
When the user searches for "Milk"	(2.824s)	features/steps/search_steps.py:6
Then search results should be displayed	(0.012s)	features/steps/search_steps.py:12
Scenario: Search for Rice		features/search.feature:10
Given the user opens the BigBasket Bangladesh website	(0.785s)	features/steps/homepage_steps.py:6
When the user searches for "Rice"	(3.057s)	features/steps/search_steps.py:6
Then search results should be displayed	(0.015s)	features/steps/search_steps.py:12
Scenario: Search for Biscuit		features/search.feature:14
Given the user opens the BigBasket Bangladesh website	(0.805s)	features/steps/homepage_steps.py:6
When the user searches for "Biscuit"	(4.996s)	features/steps/search_steps.py:6
Then search results should be displayed	(0.013s)	features/steps/search_steps.py:12
Scenario: Search for Egg		features/search.feature:18
Given the user opens the BigBasket Bangladesh website	(0.097s)	features/steps/homepage_steps.py:6
When the user searches for "Egg"	(3.616s)	features/steps/search_steps.py:6
Then search results should be displayed	(0.014s)	features/steps/search_steps.py:12
Scenario: Search for Cooking Oil		features/search.feature:22
Given the user opens the BigBasket Bangladesh website	(0.761s)	features/steps/homepage_steps.py:6
When the user searches for "Cooking Oil"	(8.063s)	features/steps/search_steps.py:6
Then search results should be displayed	(0.015s)	features/steps/search_steps.py:12

Detailed execution result of an automated test scenario.

Challenges, Solutions & Key Learning

CHALLENGE

Dynamic Web Elements

SOLUTION: Used appropriate locators and explicit waits.

CHALLENGE

Repeated UI Actions

SOLUTION: Used reusable Page Object methods.

CHALLENGE

Debugging Test Failures

SOLUTION: Used logging, screenshots and HTML reporting.

CHALLENGE

Browser Driver Management

SOLUTION: Used WebDriver Manager.

Key Learning & Future Improvements

- CI/CD pipeline integration
- Parallel test execution for faster runs
- Cross-browser test execution
- Expand coverage to checkout & payment flows

Thank You